

Mediroza General Hospital

Black Box Penetration Testing Report

Prepared by: Anieka Baxter

Cybersecurity Mentor: Waqas Karim, CCIE

Organisation: Networkwalks

Batch: B082 | Week 4 Capstone Project

Target: <https://medirozahospital.com>

Classification: **Confidential**

1. Executive Summary

A black-box penetration test was conducted against the web infrastructure of **Mediroza General Hospital** at <https://medirozahospital.com>, carried out under written authorisation from the client. The objectives of this assessment were to identify security vulnerabilities, demonstrate their potential real-world impact through controlled exploitation, and deliver actionable recommendations to strengthen the organisation's overall security posture.

The assessment uncovered seven vulnerabilities spanning a severity range from Medium to Critical. The most consequential finding was a SQL injection vulnerability present on the patient portal login page. This flaw allowed complete authentication bypass without the need for any valid credentials, granting access to three confidential patient laboratory report PDFs. The PDF encryption was subsequently defeated using password-cracking tools, and metadata embedded within one of the files revealed the existence of a forgotten database backup stored in a publicly accessible server directory. That backup contained the monthly salaries of all 30 hospital employees as well as the shareholding details of all 10 company shareholders.

The organisation's current security posture is critically deficient. The vulnerabilities identified would allow an unauthenticated external attacker to access confidential patient records, sensitive staff financial data, and private corporate ownership information using minimal effort and no specialised equipment.

Overall Risk Rating: **CRITICAL — Immediate remediation is strongly advised.**

2. Scope and Methodology

2.1 Scope

This assessment was scoped to the following target, as formally agreed with the client prior to testing:

Target Domain: <https://medirozahospital.com>

The following activities were explicitly excluded from scope: social engineering, denial-of-service attacks, and any testing directed at systems or domains outside the agreed target.

2.2 Methodology

The assessment followed a structured black-box penetration testing methodology, executed across four sequential phases:

- **Reconnaissance** — Passive information gathering using publicly available sources and web-based tools.
- **Vulnerability Identification** — Analysis of application behaviour to detect weaknesses in authentication mechanisms and input handling.
- **Exploitation** — Controlled exploitation of identified vulnerabilities to demonstrate real-world impact.
- **Documentation** — Comprehensive recording of all findings, supporting evidence, and remediation guidance in this report.

2.3 Tools Used

Tool	Purpose
waw00f	Detects the presence of a firewall
curl	Sending HTTP requests and analysing web server responses
Browser Developer Tools	Inspecting page source and login form behaviour
Networkwalks Hash Calculator	Extracting password hashes from PDF files
Networkwalks Password Cracker	Cracking PDF password hashes using wordlists

qpdf	Decrypting password-protected PDF files post-cracking
exiftool	Extracting embedded metadata from PDF files
ChatGPT	Parsing and presenting raw SQL data in readable table format

3. Findings and Proof of Exploitation

3.1 Summary of Findings

#	Vulnerability	Location	Risk
1	Username enumeration on login page	patient/login.php	Medium
2	SQL injection login bypass	patient/login.php	Critical
3	Confidential PDFs accessible after login bypass	patient/reports/	High
4	Weak PDF passwords crackable with a wordlist	patient_report_*.pdf	High
5	Sensitive metadata embedded in patient PDF files	patient_report_3.pdf	Medium

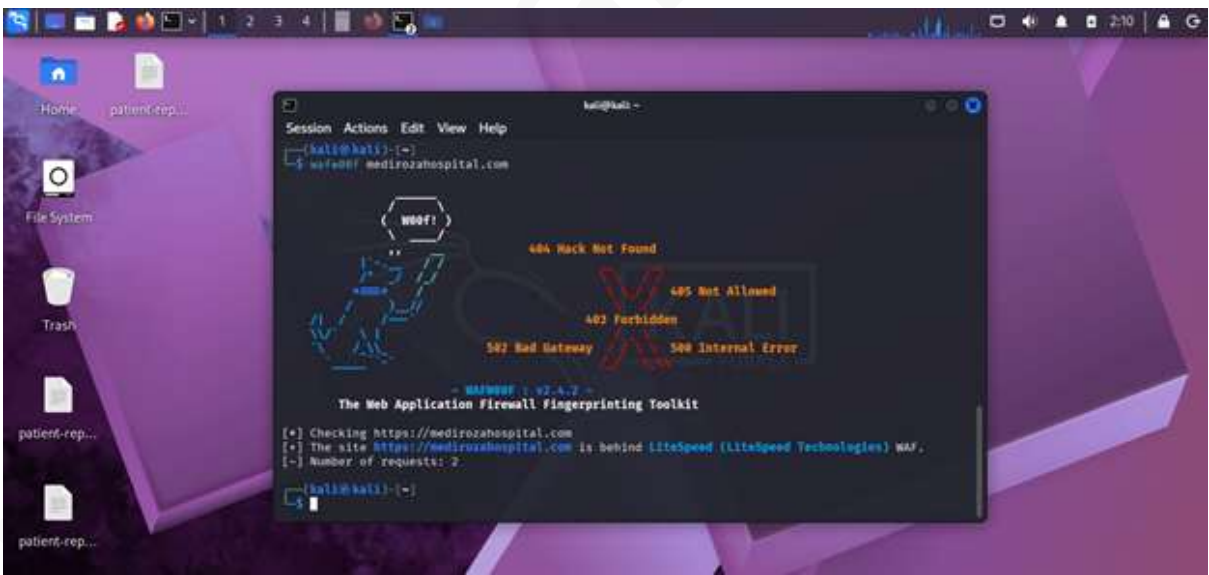
6	Forgotten backup folder with directory listing enabled	old/	Critical
7	Confidential staff salaries and shareholder data exposed in plain text	old/mediroza_db_backup_2019.sql	Critical

3.2 Reconnaissance

1. WAF Detection – WAFW00F

Risk Level: Informational

The reconnaissance identified the presence of a Web Application Firewall (WAF) protecting the Mediroza General Hospital website. The WAF is provided by **LiteSpeed Technologies** and is designed to act as a security layer between incoming web requests and the web application.



1. Robots.txt Information Disclosure

Risk Level: Low

The reconnaissance identified a publicly accessible robots.txt file containing several restricted paths:

- /patient/
- /staff/
- /old/

The robots.txt file provides information about specific directories within the Mediroza General Hospital website that the site administrator does not want search engine crawlers to access. The file identifies three paths: /patient/, /staff/, and /old/. Although these paths are marked as disallowed for web crawlers, the entries do not prevent users from accessing the directories directly.

```
medirozahospital.com/robots.txt

# robots.txt
User-agent: *
Disallow: /patient/
Disallow: /staff/
Disallow: /old/

Sitemap: https://medirozahospital.com/sitemap.xml
```

3.3 Finding 1 – Username Enumeration

Risk Rating: **Medium**

Location: patient/login.php

Description

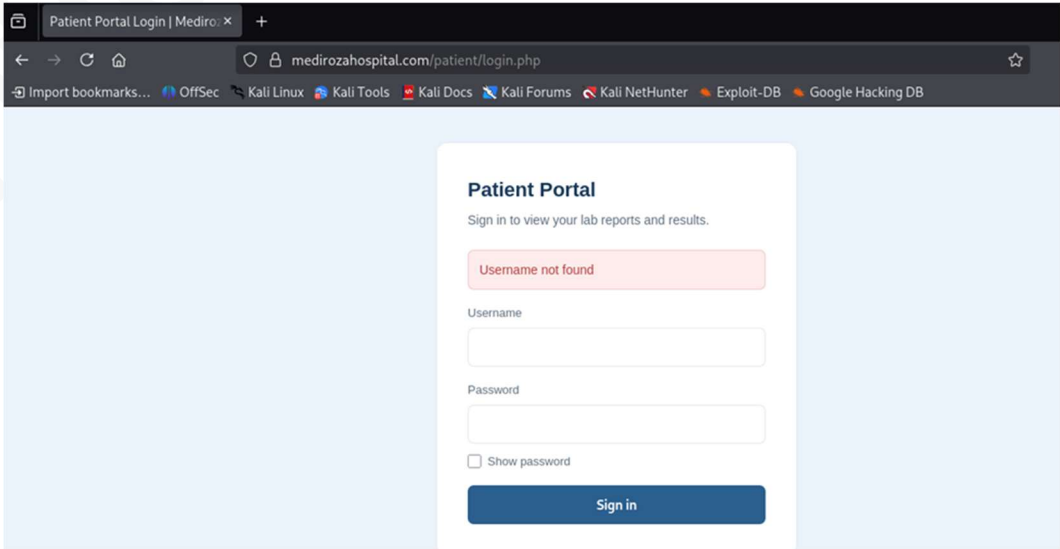
Username enumeration occurs when a login page discloses whether a submitted username exists in the system by returning distinct error messages depending on whether the username or the password was incorrect. A properly secured login page should always return a single, uniform error message for any failed attempt, regardless of the reason, to prevent an attacker from confirming the existence of valid accounts.

Steps Taken

The patient portal login page was accessed and tested with two different input combinations to observe the error message behaviour.

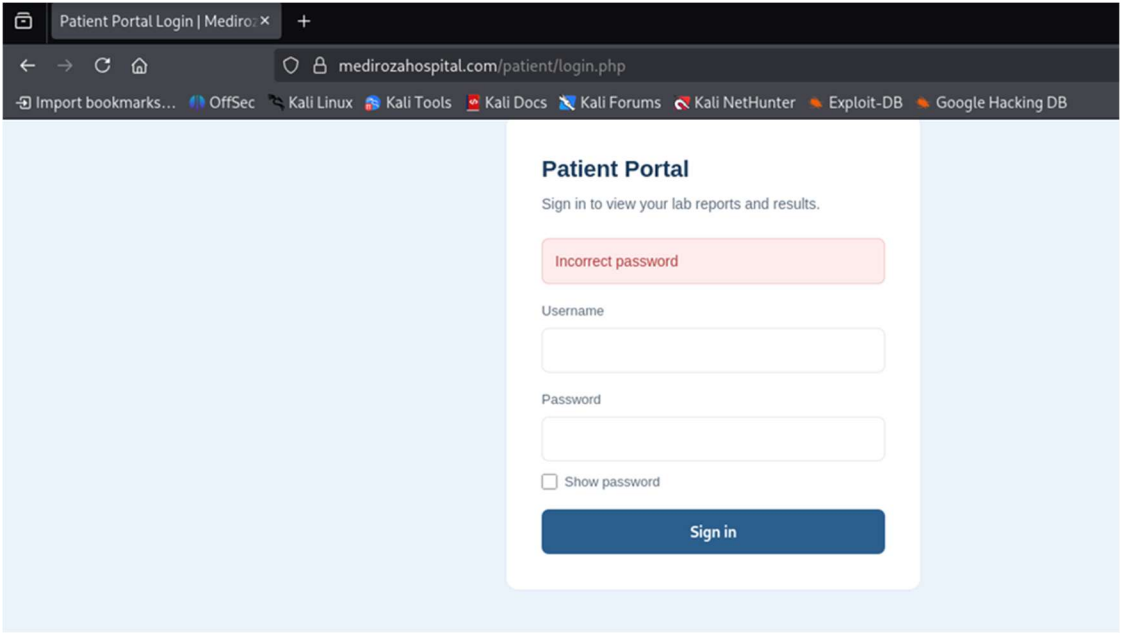
First, a username assumed to be non-existent was submitted:

- **Input:** username: alice
password: 123456
- **Response:** "Username not found"



Next, a common default username was tested:

- **Input:** username: admin
password: test123
- **Response:** "Incorrect password"



The two distinct responses confirmed that "admin" is a valid account on the system. With the username established, only the password remained unknown.

3.4 Finding 2 – SQL Injection Login Bypass

Risk Rating: Critical

Location: patient/login.php

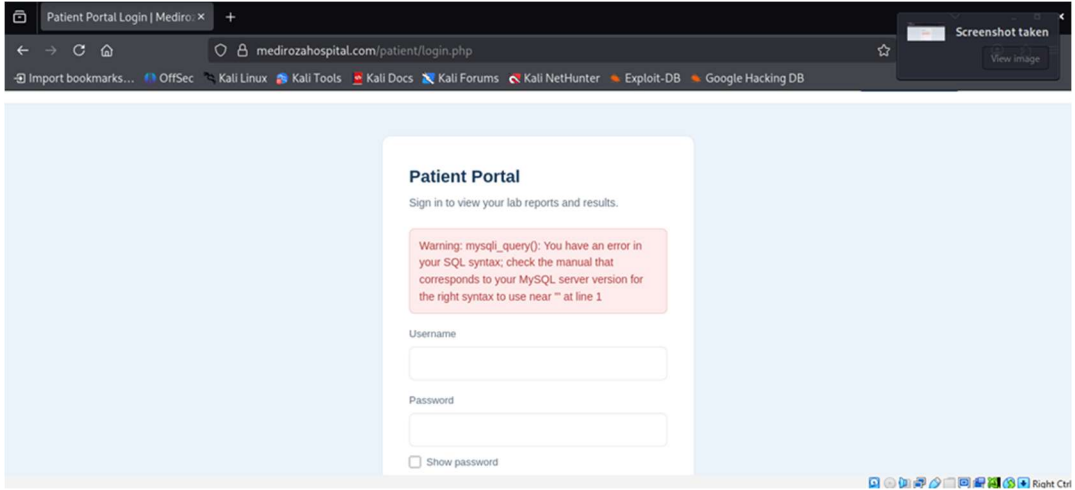
Description

SQL injection is a vulnerability that arises when an application incorporates user-supplied input directly into a database query without first sanitising or validating it. An attacker can craft malicious input to alter the intended logic of the query. In this instance, it was possible to comment out the password verification step entirely, achieving a full authentication bypass as the admin user without possessing any valid credentials.

Steps Taken

The username field was probed for SQL injection by submitting a single quotation mark:

- Input:** username: admin'
password: test123
- Response:** *Warning: mysqli_query(): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '' at line 1*



The database error confirmed that the field was injectable. The application was constructing its SQL query as follows:

```
SELECT * FROM users WHERE username='admin" AND password='test123'
```

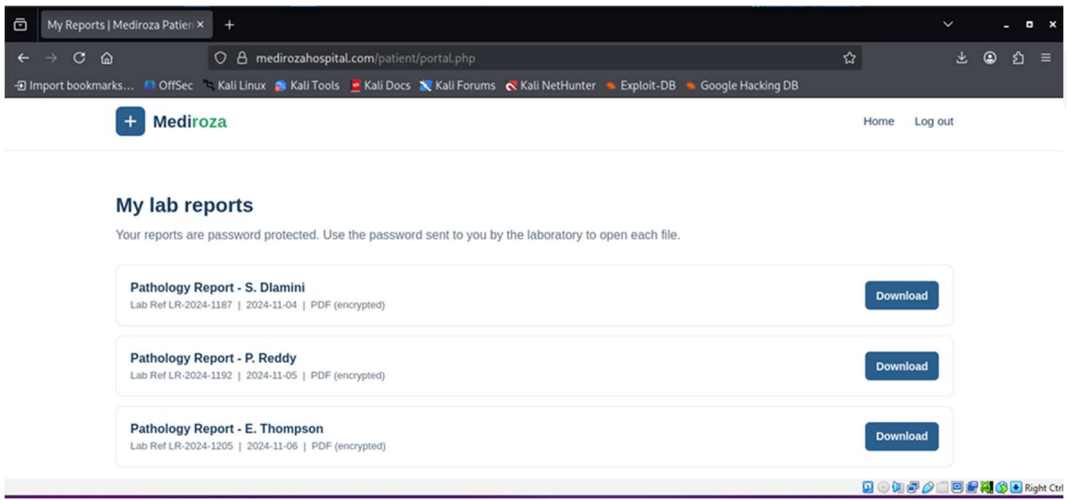
The unescaped quote broke the query syntax and triggered the error. The following classic SQL injection bypass payload was then crafted:

- **Input:** username: admin' --
password: anything

The double-dash (--) sequence comments out all subsequent SQL, reducing the query to:

```
SELECT * FROM users WHERE username='admin'
```

With the password check entirely removed, the query returned the admin record and access to the portal was granted.



3.5 Finding 3 – Confidential PDFs Accessible After Login Bypass

Risk Rating: High

Location: patient/reports/

Description

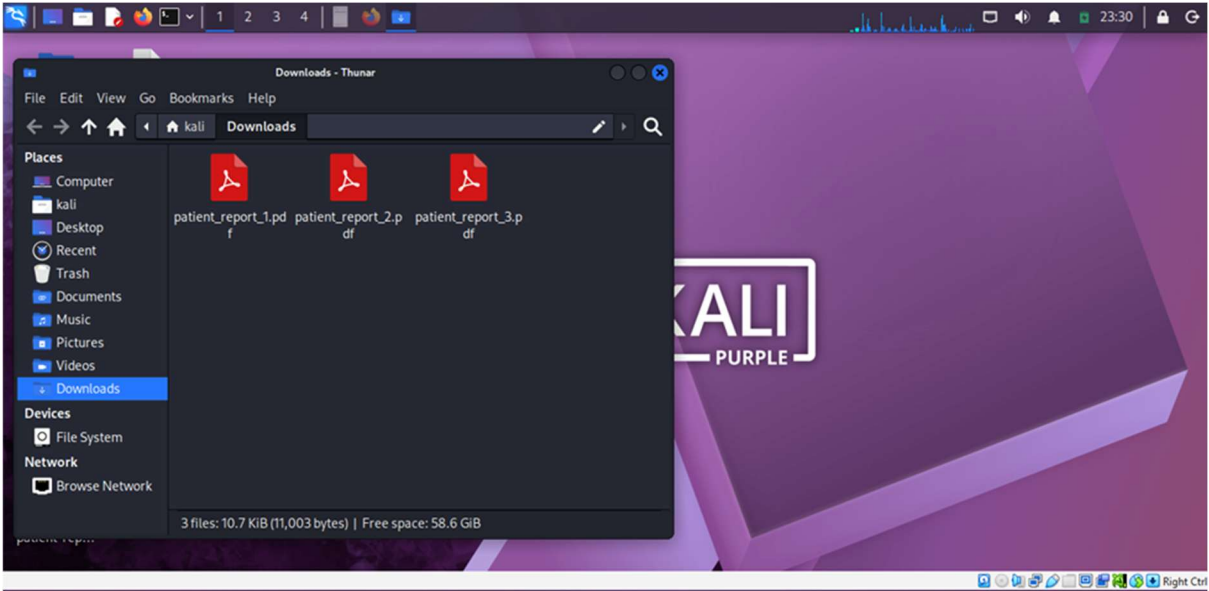
Following unauthorised access to the patient portal via SQL injection, three patient laboratory report PDFs were found available for download. These documents are confidential medical records that should be accessible only to the respective patients and their treating physicians.

Steps Taken

After authenticating via the SQL injection payload, the portal presented three downloadable files:

- patient_report_1.pdf
- patient_report_2.pdf
- patient_report_3.pdf

All three files were downloaded for further analysis.



3.6 Finding 4 — Weak PDF Passwords Crackable with a Wordlist

Risk Rating: **High**

Description

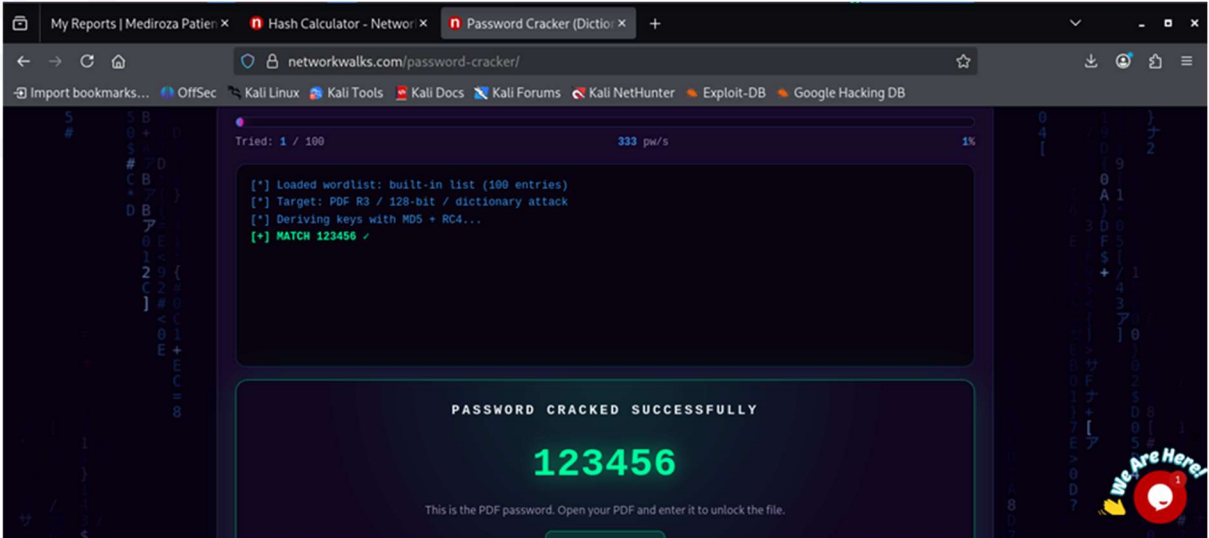
All three downloaded PDFs were password-protected. However, the passwords chosen were insufficiently complex and were present in widely available password wordlists, making them straightforward to crack using automated tools. Protecting sensitive medical documents with easily guessable passwords provides no meaningful security barrier.

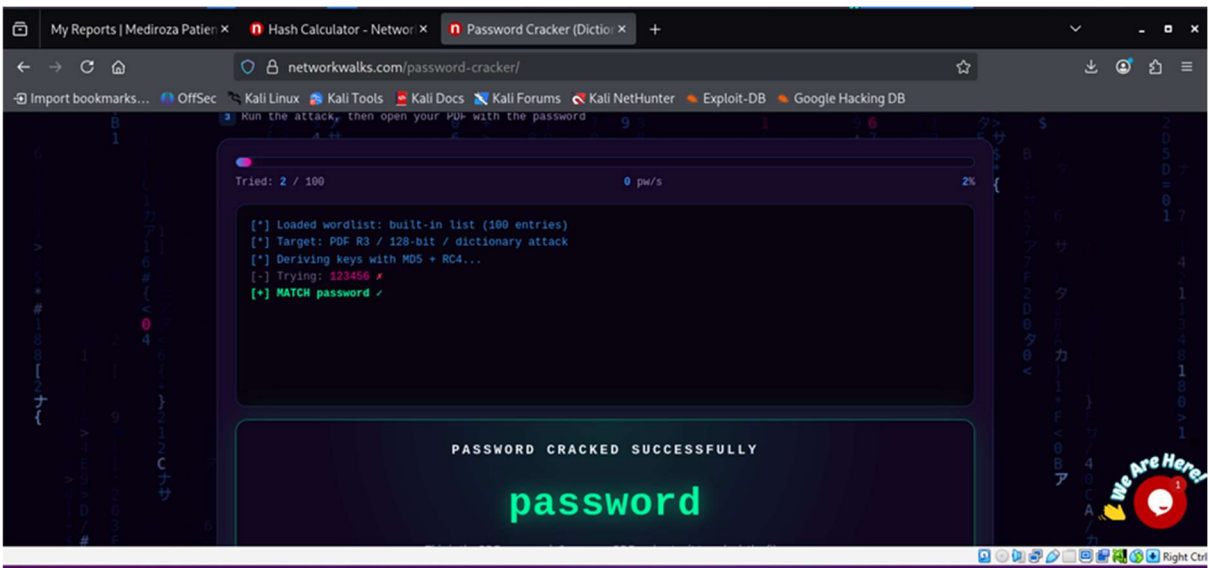
Steps Taken

The **Networkwalks Hash Calculator** was used to extract a crackable hash from each PDF file. Each hash was then submitted to the **Networkwalks Password Cracker**.

Reports 1 and 2 were cracked immediately using the built-in 100-word default wordlist:

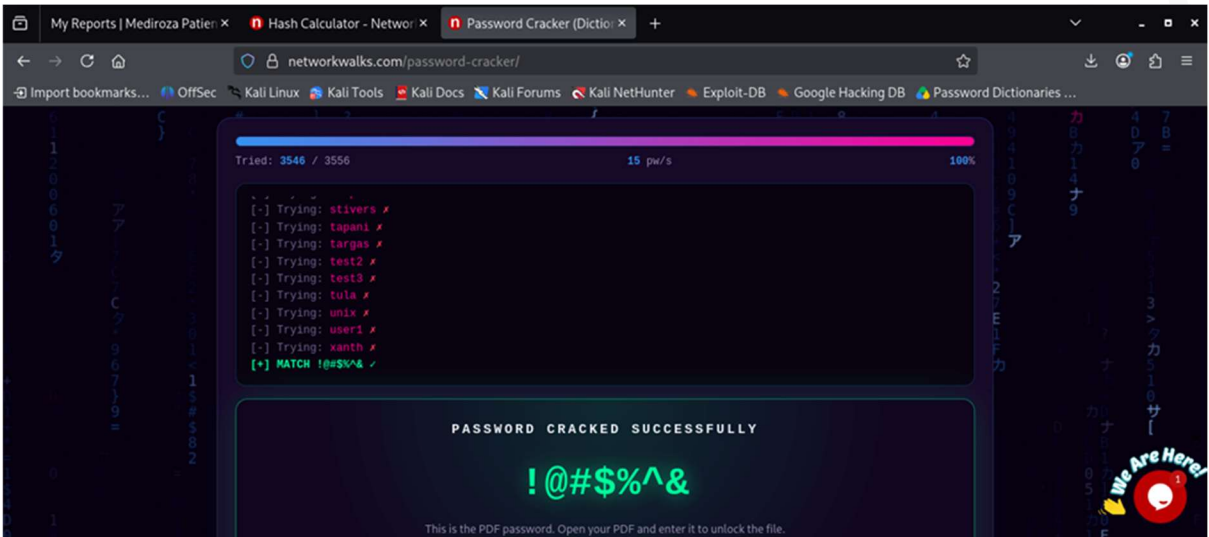
- patient_report_1.pdf → **123456**
- patient_report_2.pdf → **password**





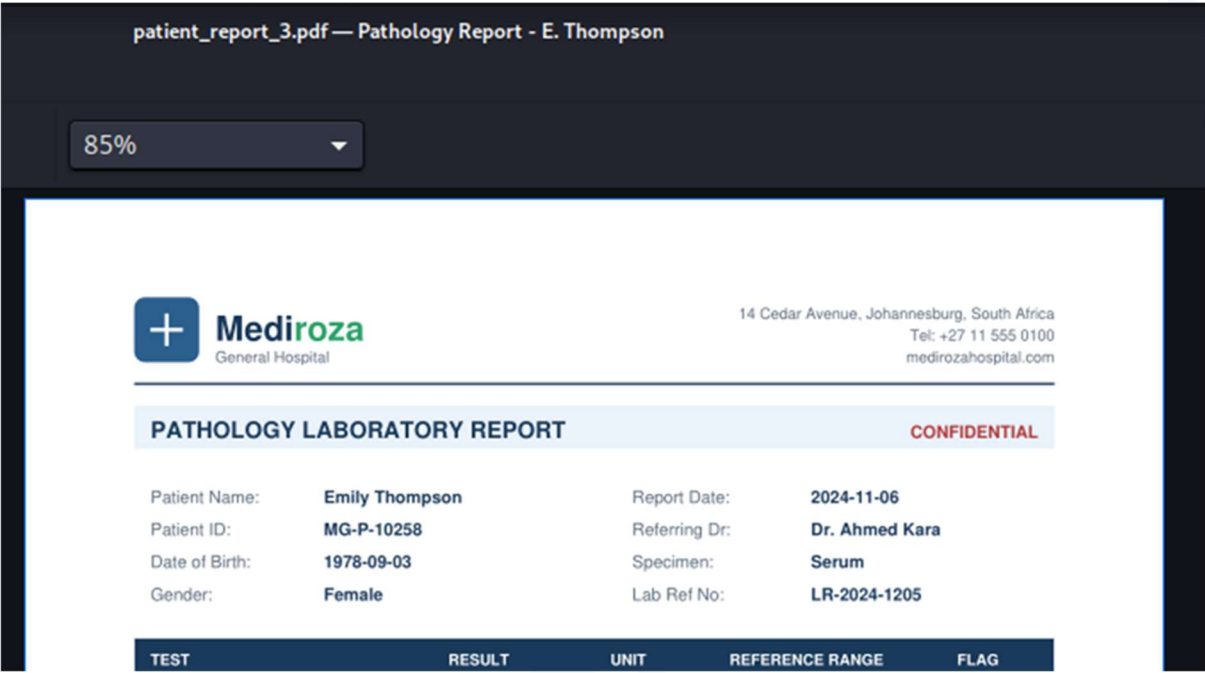
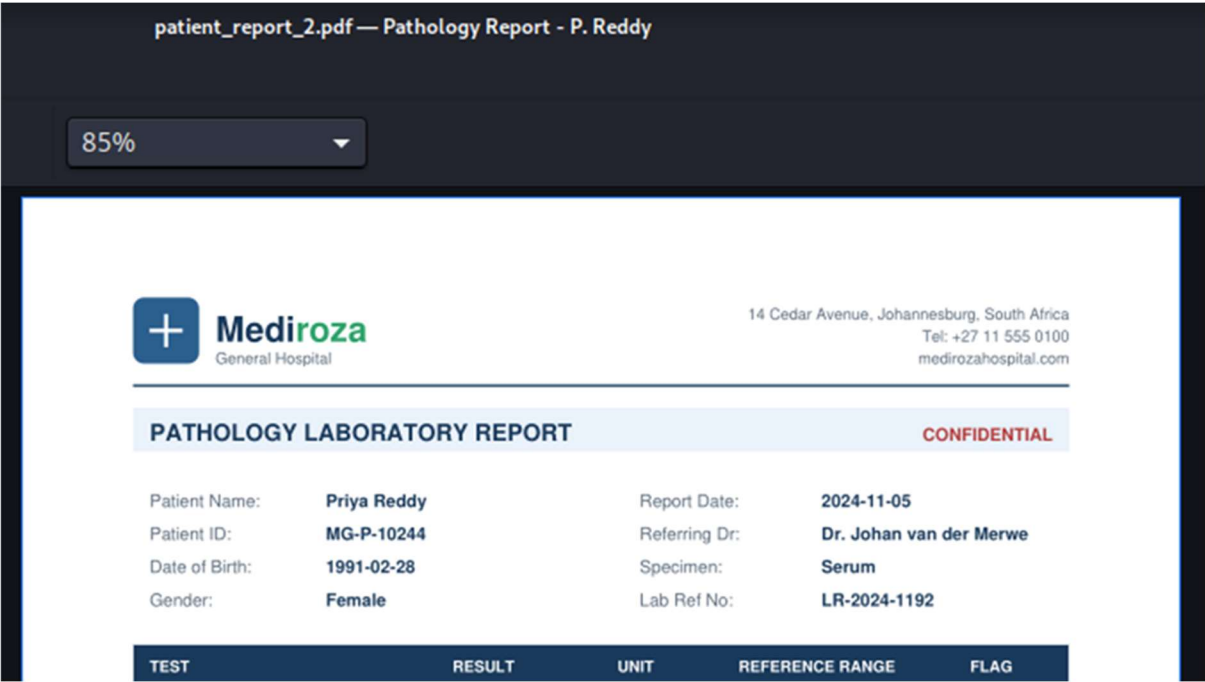
Report 3 did not yield a result with the default list. The attack was repeated using the larger JTR (John the Ripper) default wordlist:

- patient_report_3.pdf → !@#\$%^&



Each PDF was subsequently opened using its recovered password, and the confidential patient medical data contained within was accessed.





3.7 Finding 5 — Sensitive Metadata Embedded in Patient PDF Files

Risk Rating: **Medium**

Description

PDF files can contain embedded metadata fields, such as author name, creation date, and custom comments that are not visible during normal document viewing but can be extracted using tools such as **exiftool**. In this case, patient_report_3 contained an internal staff comment authored by the IT administrator, which directly disclosed the location of a confidential database backup stored on the server.

Steps Taken

Report 3 was first decrypted using **qpdf** to allow exiftool to access all metadata fields (an encrypted PDF only exposes its encryption type):

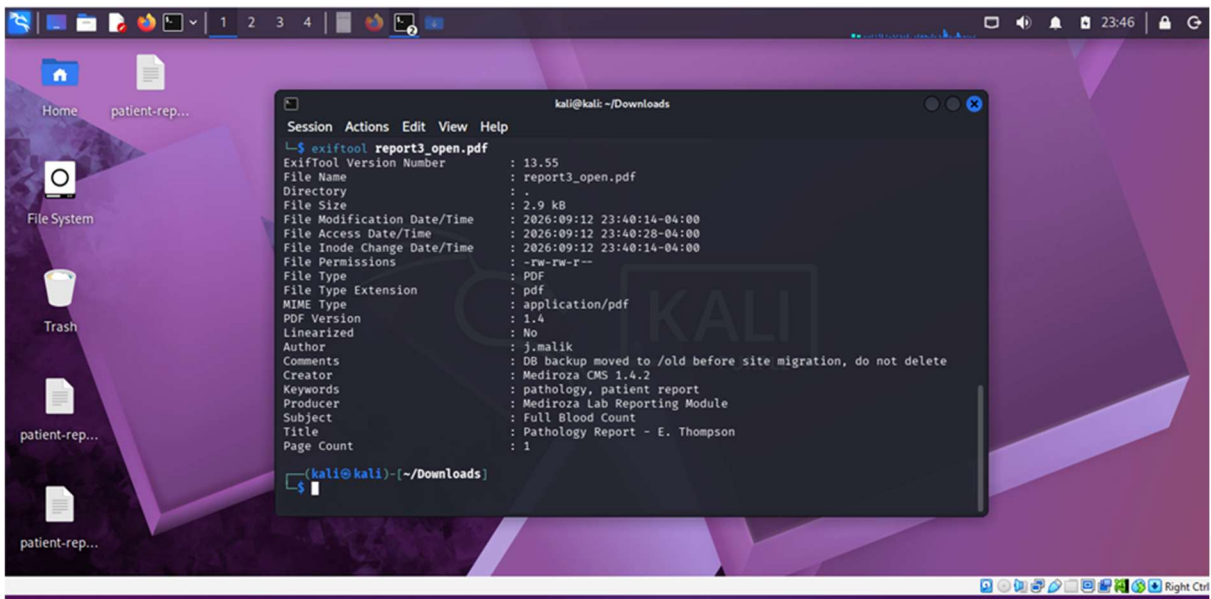
```
qpdf --password='!@#$$%^&' --decrypt patient_report_3.pdf report3_open.pdf
```

exiftool was then run against the decrypted file:

```
exiftool report3_open.pdf
```

The following key fields were present in the output:

- **Author:** j.malik
- **Comments:** DB backup moved to /old before site migration, do not delete



This comment, left by IT administrator Jameel Malik (j.malik), revealed the precise server path of a database backup file.

3.8 Finding 6 – Forgotten Backup Folder with Directory Listing Enabled

Risk Rating: Critical

Description

Directory listing is a web server misconfiguration that causes the server to display the contents of a directory as a browsable file list when no index page is present. The /old directory on this server had directory listing enabled, exposing a database backup file to any visitor who navigated to that URL. This directory had also been identified during initial reconnaissance, as it was listed as a disallowed path in the site's robots.txt file.

Steps Taken

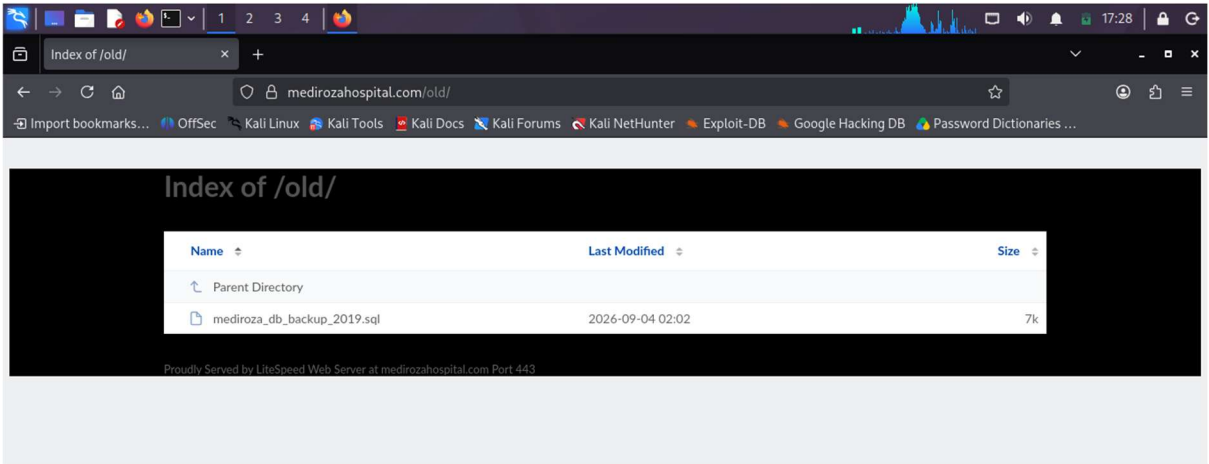
The /old path was noted during the reconnaissance phase from the robots.txt file. The metadata finding from Finding 5 corroborated this as the backup location. The directory was accessed directly in the browser:

<https://medirozahospital.com/old/>

Directory listing was active, and the following backup file was immediately visible:

- mediroza_db_backup_2019.sql

The file was retrieved by downloading it from the web server:



3.9 Finding 7 – Confidential Staff Salaries and Shareholder Data Exposed in Plain Text

Risk Rating: **Critical**

Description

The downloaded database backup contained highly sensitive personal and financial information stored entirely in plain text. This included the full name, job title, email address, phone number, national ID number, and monthly salary (in ZAR) for all 30 hospital employees, as well as the names and shareholding percentages of all 10 hospital shareholders. This data was completely unprotected and accessible to any individual who visited the /old directory.

Steps Taken

The SQL file was opened in a text editor. The data was in raw SQL INSERT format. The staff records were copied and submitted to ChatGPT with the following prompt:

"Present this SQL data as a readable table showing name, job title, department, and monthly salary."

The same process was repeated for the shareholder records:

"Present this SQL data as a readable table showing shareholder name, share percentage, and share class."

The private salary of every employee and the complete ownership structure of the hospital were successfully retrieved and rendered in readable form.

Additionally, the Author field identified in Finding 5 – j.malik – was confirmed to correspond to Jameel Malik, the IT Systems Administrator listed in the staff table. This connection links all three exploitation milestones into a single, continuous attack chain.

1. Staff Table

Share

ID	Full Name	Job Title	Department	Email	Phone	National ID	Monthly Salary (ZAR)	Date Joined
1	Dr. Rajesh Naidoo	Chief Pathologist	Diagnostics Lab	r.naidoo@medirozahospital.com ✉	+27 82 101 2007	85021013011081	138,000	2009-03-16
2	Sarah Botha	Chief Financial Officer	Finance	s.botha@medirozahospital.com ✉	+27 82 102 2014	85031023022082	152,000	2011-07-01
3	Dr. Johan van der Merwe	Medical Director	Management	j.merwe@medirozahospital.com ✉	+27 82 103 2021	85041033033083	160,000	2007-01-22
4	Dr. Anita Naicker	Consultant Cardiologist	Cardiology	a.naicker@medirozahospital.com ✉	+27 82 104 2028	85051043044084	132,000	2012-09-10
5	Dr. Ahmed Kara	Consultant Physician	Internal Medicine	a.kara@medirozahospital.com ✉	+27 82 105 2035	85061053055085	128,000	2013-02-18
6	Dr. Yusuf Cassim	Senior Registrar	Emergency & Trauma	y.cassim@medirozahospital.com ✉	+27 82 106 2042	85071063066086	74,000	2018-05-04
7	Michael Roberts	HR Director	Human Resources	m.roberts@medirozahospital.com ✉	+27 82 107 2049	85081073077087	96,000	2010-11-15
8	Susan Pretorius	HR Officer	Human Resources	s.pretorius@medirozahospital.com ✉	+27 82 108 2056	85091083088088	32,000	2016-08-23
9	Jameel Malik	IT Systems Administrator	IT	j.malik@medirozahospital.com ✉	+27 82 109 2063	85101093099080	58,000	2015-04-12
10	Thabo Molefe	Network Engineer	IT	t.molefe@medirozahospital.com ✉	+27 82 110 2070	85111103110081	46,000	2017-10-02
11	Nomvula Khumalo	Registered Nurse	Emergency & Trauma	n.khumalo@medirozahospital.com ✉	+27 82 111 2077	85121113121082	34,000	2016-01-19
12	Lerato Mokoena	Registered Nurse	Pediatrics	l.mokoena@medirozahospital.com ✉	+27 82 112 2084	85011123132083	33,000	2017-06-07
13	Bongani Ndlovu	Registered Nurse	Cardiology	b.ndlovu@medirozahospital.com ✉	+27 82 113 2091	85021133143084	35,000	2015-12-01
14	Zanele Mahlangu	Nursing Sister	Theatre	z.mahlangu@medirozahospital.com ✉	+27 82 114 2098	85031143154085	42,000	2013-03-25
15	Kagiso Sithole	Pharmacist	Pharmacy	k.sithole@medirozahospital.com ✉	+27 82 115 2105	85041153165086	61,000	2014-09-08
16	Naledi Zulu	Pharmacy Assistant	Pharmacy	n.zulu@medirozahospital.com ✉	+27 82 116 2112	85051163176087	26,000	2019-02-14

2. Shareholders Table

Copy

ID	Shareholder Name	Share %	Shares Held	Share Class
1	Dr. Rajesh Naidoo	18.00%	180,000	Ordinary
2	Cedar Health Holdings (Pty) Ltd	15.00%	150,000	Ordinary
3	Dr. Johan van der Merwe	12.00%	120,000	Ordinary
4	Reddy Family Trust	11.00%	110,000	Ordinary
5	Thabo Molefe	10.00%	100,000	Ordinary
6	Sarah Botha	9.00%	90,000	Ordinary
7	Dr. Ahmed Kara	8.00%	80,000	Preferential
8	Naledi Zulu	7.00%	70,000	Ordinary
9	Michael Roberts	6.00%	60,000	Ordinary
10	Dr. Vikram Chetty	4.00%	40,000	Preferential

4. Full Attack Chain Summary

The following outlines the complete exploitation path from zero access to full data exposure, illustrating how each finding enabled the next.

1. **Reconnaissance** — The robots.txt file disclosed three hidden directories: /patient, /staff, and /old.
 2. **Username Enumeration (Finding 1)** — Distinct login error messages confirmed "admin" as a valid account.
 3. **SQL Injection Confirmed (Finding 2)** — A single quote in the username field triggered a database error, confirming the field was vulnerable to SQL injection.
 4. **Authentication Bypass (Finding 2)** — The payload admin' -- bypassed the login entirely, granting access to the patient portal.
 5. **PDF Download (Finding 3)** — Three confidential patient lab report PDFs were downloaded from the portal.
 6. **Password Cracking (Finding 4)** — All three PDF passwords were cracked using wordlists; Reports 1 and 2 with the default list, Report 3 with the JTR list.
 7. **Metadata Extraction (Finding 5)** — exiftool on the decrypted Report 3 revealed a hidden staff comment pointing to the /old directory.
 8. **Directory Exposure (Finding 6)** — The /old folder had directory listing enabled, exposing the database backup file.
 9. **Data Exfiltration (Finding 7)** — The backup contained the salaries of 30 employees and the shareholding details of 10 shareholders, all in plain text.
-

5. Recommendations and Remediation

5.1 Remediate Username Enumeration

Update the login page to return a single, uniform error message for all failed login attempts, regardless of whether the username or the password was the cause of the failure. A suitable message would be: *"Invalid credentials. Please try again."* or *"Invalid username or password."* This prevents attackers from using the login page to confirm which usernames are registered on the system.

5.2 Remediate SQL Injection

Replace the existing login query with a parameterised query or prepared statement. This approach separates SQL logic from user-supplied input, ensuring that no crafted input can alter the structure or behaviour of the query. **This is the highest-priority remediation item in this report.**

A safe implementation example using PHP PDO:

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ? AND password = ?");  
$stmt->execute([$username, $password]);
```

5.3 Remediate PDF Access Controls and Password Strength

Relocate all PDF files to a directory outside the web root so they cannot be served directly by the web server. Implement a server-side authentication check that verifies a user's session before delivering any file. Where PDF passwords are used, enforce a password policy - minimum length of 12 characters incorporating a combination of uppercase letters, lowercase letters, numbers, and special characters.

5.4 Strip Metadata from PDF Files Before Distribution

All metadata must be removed from documents prior to distribution. Internal staff notes and comments must never be stored within files that leave the organisation's internal systems. The following command removes all metadata from a PDF:

```
exiftool -all= patient_report_3.pdf
```

This process should be incorporated into any document workflow that involves external distribution.

5.5 Disable Directory Listing and Remove the Exposed Backup

Directory listing must be disabled across all server directories. This can be achieved by adding the following directive to the Apache configuration or the relevant .htaccess file:

Options -Indexes

The database backup file must be deleted from the /old directory immediately. Database backups must never be stored within the web root. All backups should be retained in a private, access-controlled location that is entirely separate from the publicly accessible portion of the server.

6. Conclusion

This assessment identified a complete, end-to-end attack chain leading from the public-facing login page to the exposure of highly sensitive internal data. No advanced tools or specialised expertise were required to execute the full chain. Every vulnerability documented in this report is well understood within.

Submitted by: Anieka Baxter

Cybersecurity Mentor: Waqas Karim, CCIE

Organisation: Networkwalks

Batch B082 | Week 4 Capstone Project